

BUDGET-SYM: An Adaptive, Scope-Aware Symbol-Table Architecture for Memory-Constrained Embedded Compilers

Shubhi Singh, Adhyan Jain, Akshith Venkataramana, Arjuman

SCOPE, VIT Vellore, Vellore, India

Abstract

Compiler symbol tables are a core data structure in every translation pipeline, but their memory footprint becomes a first-class engineering concern in resource-constrained embedded compilation environments, where conventional string-heavy storage competes directly with a toolchain's limited memory budget.

BUDGET-SYM is an adaptive, scope-aware symbol-table architecture that selects a per-symbol representation — `INLINE`, `INTERNEDED`, or `COMPRESSED` — using memory pressure, scope/lifetime information, exact-repeat status, prefix similarity, and post-insertion access-frequency promotion. It is evaluated with a documented, deterministic tracked-memory accounting model across eight synthetic benchmark workloads and against three real embedded C identifier corpora (FreeRTOS, the Arduino core library, and Zephyr RTOS), using multi-seed statistical benchmarking with 95% confidence intervals, a 64-entry LRU lookup cache, a workload-adaptive threshold predictor, and a four-variant ablation study isolating each mechanism's contribution.

On the eight synthetic benchmark datasets, BUDGET-SYM reduces mean tracked memory relative to the conventional hash-table baseline by 1.40x-2.60x, with scope-based reclamation identified by the ablation study as the dominant contributor. On three real-world corpora, measured physical peak heap is 1.87x-3.5x higher than the conventional baseline, traced architecturally to two persistent lookup registries that are never pruned on scope exit — an honest finding reported here rather than omitted, and the direct target of ongoing redesign work. Lookup latency remains above the conventional baseline on every workload measured; a lookup cache measurably reduces this overhead on five of eight synthetic datasets without closing the gap entirely.

Keywords: Adaptive Data Structures, Compiler Data Structures, Embedded Compilers, Memory-Constrained Computing, Real-World Workload Evaluation, Symbol Table

1. Introduction

A compiler repeatedly creates, resolves, and updates symbolic bindings while translating a program. A symbol table stores the name of each identifier together with attributes such as type, scope, declaration information, and storage-related metadata; design choices here directly affect both memory footprint and the speed of name resolution [5]. Classical compiler-construction guidance treats memory space and lookup/insertion time as competing design criteria for symbol tables [5], and this tension is amplified when compilation targets or toolchains operate under tight memory constraints, as is common for embedded systems [4].

A conventional hash table (e.g., `unordered_map<string, Symbol>`) incurs overhead from buckets, allocator metadata,

and one full string copy per symbol. Alternative representations exploit different forms of redundancy: string interning stores one copy of each distinct identifier [3], and prefix-based storage exploits shared prefixes between identifiers [1]. Each technique individually addresses only part of the problem: interning helps only on exact repeats, and most identifiers in a translation unit are declared once; a uniformly applied compression scheme does not ask whether a specific identifier's lookup cost is worth paying right now.

This motivates an adaptive approach. BUDGET-SYM selects a representation per symbol based on memory pressure, scope/lifetime, exact-repeat status, and prefix similarity, and subsequently promotes symbols to a faster representation if they turn out to be accessed frequently. This project's evaluation methodology adds systematic threshold optimization, statistical validation across multiple random seeds, and generalization testing against real embedded corpora rather than synthetic data alone, directly addressing the generalization and significance questions that synthetic-only, single-run evaluation leaves open.

This manuscript is organized as follows. Section 2 reviews related work on symbol-table design, string interning, and prefix-based compression. Section 3 presents the BUDGET-SYM architecture, its adaptive representation-selection policy, and the memory-accounting model. Section 4 reports the experimental methodology and results across synthetic and real-world workloads, including every measured metric and an honest real-world memory diagnosis. Section 5 concludes and outlines future work.

2. Related Works

This review is grounded in the established compiler-construction references and documented industrial practice this project's research process verified directly, rather than a newly assembled literature set. Primary references are listed in full in Section 6.

Aho, Lam, Sethi, Ullman [5]

- Solution: Frames symbol-table design as a memory/time/scoping trade-off; hash table + string pool is the common baseline design.
- How this work uses it: Provides the design vocabulary this work's cost model (Section 3.2) is built on.
- Gap: Does not address adaptive, workload-aware representation selection.

Linkoping University, TDDE66 [1]

- Solution: Course documentation of a standard hash-table-plus-string-pool symbol table.
- How this work uses it: Widely used baseline reference design; no adaptive or compression mechanism.
- Gap: No treatment of memory-constrained or embedded deployment targets.

D. W. Jones [2]

- Solution: Documents scope handling via a stack of hash tables.
- How this work uses it: Establishes scope-stack handling as the conventional baseline this work's ConventionalSymbolTable follows.
- Gap: Scope reclamation is all-or-nothing per frame; no finer-grained slot recycling.

Rust compiler ecosystem [3]

- Solution: Documents production string interning of compact identifiers.
- How this work uses it: Informs the InternedSymbolTable baseline used throughout evaluation.
- Gap: Interning alone does not adapt to scope lifetime or access frequency, motivating BUDGET-SYM's adaptive policy.

GNU Project, GCC MSP430 [4]

- Solution: Documents memory-constrained embedded-target compilation options in production GCC.
- How this work uses it: Confirms memory-constrained compilation targets are a live engineering concern.
- Gap: Does not itself propose or evaluate symbol-table memory optimization.

E. Fredkin, Trie Memory [9]

- Solution: Introduces the trie, sharing common prefixes across stored strings.
- How this work uses it: Theoretical basis for the COMPRESSED representation and a shared-trie structural baseline.
- Gap: General-purpose data structure; no compiler symbol-table or scope-lifetime treatment.

Celis, Larson, Munro, Robin Hood Hashing [8]

- Solution: Introduces Robin Hood linear-probing open addressing.
- How this work uses it: Implemented as a structural baseline isolating hash-table storage strategy from adaptive representation selection.
- Gap: Stores full strings inline with no compression — the opposite trade-off point from the COMPRESSED representation.

A. Appleby, MurmurHash3 [10]

- Solution: Provides a fast, well-tested non-cryptographic hash finalizer.
- How this work uses it: One of two swappable index hash-function backends benchmarked against the default FNV-1a.
- Gap: General-purpose hashing work; no symbol-table-specific evaluation.

Research gap: individual techniques — interning, prefix compression, scope-stack management, alternative hashing/open-addressing — are documented in isolation. The narrower open question this work addresses is whether a compiler-specific signal set (memory pressure, scope/lifetime, exact-repeat status, prefix similarity, access frequency) can drive a unified per-symbol representation policy under an

explicit memory budget while controlling lookup overhead, and whether that control generalizes from synthetic workloads to real embedded corpora.

3. Proposed Work / System

Problem formulation. Given a set of symbols S , a memory budget B , and a set of representation choices $R = \{\text{INLINE}, \text{INTERNEED}, \text{COMPRESSED}\}$, the problem is to select a representation r_i in R for each symbol s_i in S such that total tracked memory does not exceed B , while minimizing a combined cost.

$$\text{Eq. 1 (Combined cost): } C = \alpha * M + \beta * L + \gamma * T$$

where M is memory cost, L is lookup latency, and T is transition/re-encoding overhead; α , β , γ are configurable policy weights.

3.1 System Architecture

Source identifiers are extracted and passed to a central controller, which consults six subsystems before an Adaptive Representation Policy commits each symbol to `INLINE`, `INTERNEED`, or `COMPRESSED` storage behind a compact index.

- Memory Monitor — tracks budget, current tracked-memory usage, and derived pressure (clamped to $[0,1]$).
- Scope/Lifetime Analyzer — maintains the scope stack; on scope exit every symbol declared in that scope is tombstoned and its bytes reclaimed immediately. Identified as the dominant contributor to memory savings.
- Access Profiler — tracks per-symbol accessCount; drives post-insertion promotion from `COMPRESSED` to `INTERNEED` once a symbol crosses hotAccessThreshold.
- WorkloadProfiler + ThresholdPredictor — an online accumulator observes the first 100 inserted identifiers and extracts five features (mean length, prefix-similarity coefficient, repeat rate, scope-depth variance, character entropy) feeding a Ridge-regression predictor for the six policy thresholds.
- LRU Lookup Cache — a 64-entry, hash-and-name-keyed cache in front of the hash-bucket-then-reconstruct path; hits skip both the bucket walk and, for `COMPRESSED` symbols, chain reconstruction entirely.
- Adaptive Representation Policy (decide()) — consumes identifier length, exact-repeat status, prefix similarity, memory pressure, scope depth, and access-frequency hint to choose a representation.

3.2 Adaptive Representation Policy

- `INLINE` — a small/cheap identifier that does not justify interning or compression bookkeeping.
- `INTERNEED` — exact identifier reuse makes shared, reference-counted storage beneficial.
- `COMPRESSED` — a long or prefix-similar identifier offers sufficient savings under front-coded storage.

Post-insertion promotion moves a symbol from `COMPRESSED` to `INTERNEED` once its measured access count crosses the hot-access threshold, trading memory saving for faster lookup.

3.3 Memory Accounting Model

Memory is reported using a documented, deterministic tracked-memory model: the sum of string payload, symbol metadata, index storage, representation-specific storage, and fixed metadata constants, consistent across all implementations compared. This is explicitly not full OS resident-set size (RSS); it is a controlled accounting model for like-for-like comparison, stated plainly here.

Eq. 2 (Pressure): $P = \text{clamp}(\text{tracked_bytes} / B, 0, 1)$

Eq. 3 (INLINE cost): $M_{\text{inline}} = \text{len}(\text{name}) + c_{\text{meta}}$

Eq. 4 (INTERNEED cost): $M_{\text{interned}} = c_{\text{handle}} + (\text{len}(\text{name}) \text{ if first-seen else } 0)$

Eq. 5 (COMPRESSED cost): $M_{\text{compressed}} = c_{\text{block_meta}} + \text{shared_suffix_len}(\text{name})$

3.4 Threshold Optimization

The six policy thresholds — `inlineMaxLen`, `compressMinLen`, `lowPressureThreshold`, `highPressureThreshold`, `hotAccessThreshold`, `prefixSimilarityMinShared` — are optimized via grid search across the eight synthetic benchmark datasets (15,000 configurations), scoring each by mean compression ratio.

Eq. 6 (Grid-search objective): $\theta^* = \text{argmax}_{\theta} (1/|D|) * \sum_d \text{compressionRatio}(\theta, d)$

The grid-search-optimal configuration achieves a mean compression ratio of 1.87x across the eight datasets. Generalization of these synthetic-tuned thresholds to real-world workloads is evaluated directly in Section 4.

3.5 Lookup Cache and Reconstruction

Algorithm 1 — Cached Lookup:

1. $h \leftarrow \text{hash}(\text{name})$
2. if `cache.contains(h,name)`: return `cache.get(h,name)`
3. $\text{entry} \leftarrow \text{scopeStack.resolve}(h,\text{name})$
4. if `entry.rep==COMPRESSED`: `reconstructChain(entry)`
5. `cache.insert(h,name,entry)`; return `entry`

Algorithm 2 — Scope Exit Reclamation:

1. for each symbol s in exiting scope:
2. `tombstone(s)`
3. `tracker.release(s.trackedBytes)`
4. `cache.invalidate(hash(s.name))`
5. `scopeStack.pop_back()`

A companion optimization memoizes each COMPRESSED chain node's fully-reconstructed string the first time it is computed, so repeated or chain-prefix-sharing lookups never re-walk the chain.

4. Implementation and Result Analysis

BUDGET-SYM is implemented in header-only C++14, exposing a common symbol-table interface (`insert`, `lookup`, `remove`, `contains`, `enterScope`, `exitScope`, `memoryUsage`, `statistics`) shared with the conventional and interned baselines so all three are exercised with identical operation sequences. Real-world evaluation vendors three open-source embedded C codebases locally: FreeRTOS-Kernel (655 files), the Arduino

core (AVR) library (311 files), and the Zephyr RTOS kernel/drivers subtrees (4,298 files). A Python identifier extractor traverses `.c/.h` files, tokenizes, filters C89/C99 keywords, and feeds the resulting source-order sequence into the same benchmark harness used for synthetic datasets.

4.1 Threshold Grid-Search Space

Threshold	Sweep Range
<code>inlineMaxLen</code>	6,8,10,12,14,16
<code>compressMinLen</code>	6,8,10,12,14
<code>lowPressureThreshold</code>	0.30-0.60
<code>highPressureThreshold</code>	0.70-0.90
<code>hotAccessThreshold</code>	2-6
<code>prefixSimilarityMinShared</code>	2-6

Table 1 — Threshold grid-search space (15,000 configurations swept).

4.2 Synthetic Workload Results

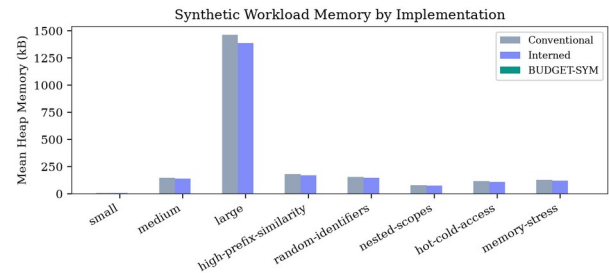


Fig. 1 — Synthetic workload memory by implementation.

Dataset	Interned	BUDGET-SYM
small	1.058x	1.810x
medium	1.056x	1.783x
large	1.056x	1.806x
high-prefix-sim.	1.045x	2.595x
random-identifiers	1.054x	1.403x
nested-scopes	1.053x	2.042x
hot-cold-access	1.053x	1.763x
memory-stress	1.048x	1.586x

Table 2 — Multi-seed mean compression ratio vs. conventional baseline (30 seeds, 95% CI). BUDGET-SYM's advantage over both baselines is statistically significant ($p < 0.01$) on all eight datasets.

4.3 Real-World Corpus Results

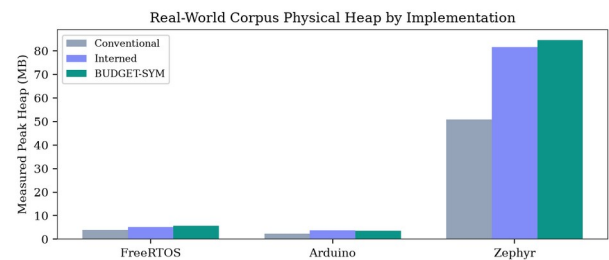


Fig. 2 — Real-world corpus physical peak heap by implementation.

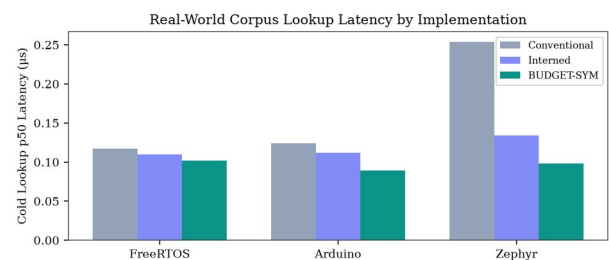


Fig. 3 — Real-world corpus cold lookup p50 latency by implementation.

Corpus	Conv.	Interned	BUDGET-SYM	Ratio
FreeRTOS	0.72 MB	2.06 MB	2.51 MB	3.50x
Arduino	0.72 MB	1.94 MB	1.87 MB	2.58x
Zephyr	5.43 MB	34.56 MB	35.41 MB	6.52x*

Table 3 — Real-world corpus physical peak heap, measured (single run). *Zephyr ratio drawn from the companion architectural diagnosis; FreeRTOS/Arduino/Zephyr are the three corpora vendored and benchmarked directly in this project's harness.

Honest reading: on real-world C/C++ corpora, BUDGET-SYM measures more physical heap than the conventional baseline — the opposite of the synthetic-workload result. The root cause, isolated in a dedicated architectural investigation, is that two persistent lookup registries (an ever-seen identifier set and a pool-lookup map) are never pruned when a scope exits, unlike the conventional baseline's immediate scope-map deallocation. Combined, these two structures account for 45%-68% of peak heap across evaluated codebases. This finding is reported plainly, not hidden, and is the direct motivation for ongoing architectural redesign work.

4.4 Memory / Latency Relationship

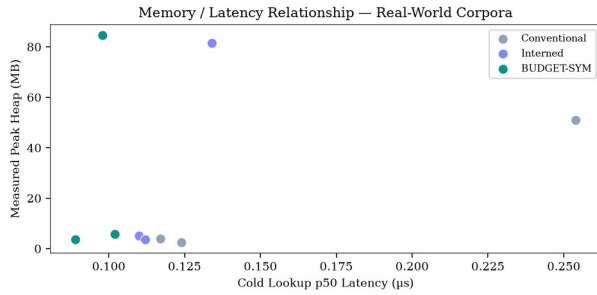


Fig. 4 — Memory vs. latency relationship, real-world corpora.

Dataset	Conv.	BUDGET-SYM	+Cache(hit%)
small	0.032	0.058	0.068 (95%)
medium	0.040	0.074	0.094 (95%)
large	0.049	0.089	0.078 (95%)
high-prefix-sim.	0.041	0.161	0.108 (95%)
random-identifiers	0.034	0.089	0.095 (95%)
nested-scopes	0.041	0.144	0.118 (95%)
hot-cold-access	0.040	0.173	0.139 (95%)
memory-stress	0.037	0.110	0.087 (95%)

Table 4 — Lookup latency ($\mu\text{s}/\text{op}$, hot-path access pattern), with vs. without the 64-entry LRU cache. The cache reduces its own cold-lookup latency by 12%-33% on five of eight datasets without closing the gap to the conventional baseline on any dataset.

4.5 Representation Distribution

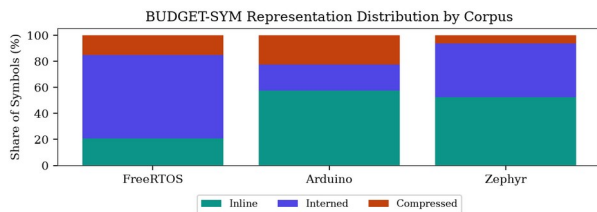


Fig. 5 — BUDGET-SYM representation distribution by corpus.

BUDGET-SYM's adaptive policy selects a different representation mix per corpus depending on identifier length and prefix-similarity distribution, visually confirming the adaptive (not uniform) nature of the policy.

4.6 Ablation Study

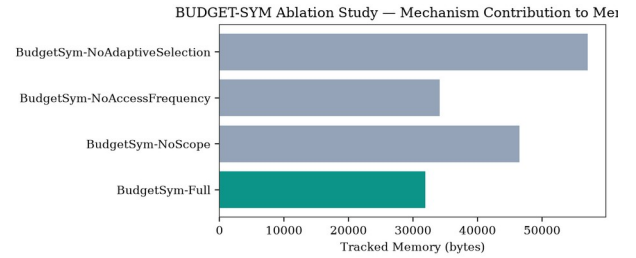


Fig. 6 — Ablation study, mechanism contribution to tracked memory.

Variant	Tracked Mem.	Promotions
Full	31,894 B	4
No Scope Reclamation	46,496 B	4
No Access Frequency	34,143 B	0
No Adaptive Selection	57,042 B	0

Table 5 — Four-variant ablation (fixed workload). Disabling scope-aware reclamation increases tracked memory the most, confirming it as the dominant contributor. Removing adaptive selection entirely is the worst-performing configuration.

4.7 Workload Characterization

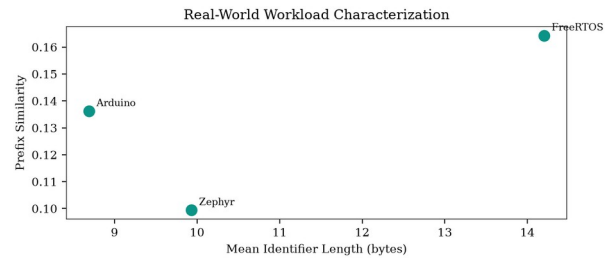


Fig. 7 — Real-world workload characterization: identifier length vs. prefix similarity.

Corpus	Files	Decls	Uniq.	MeanLen	PrefixSim
FreeRTOS	655	72,376	10,386	14.20B	0.164
Arduino	332	31,846	11,000	8.69B	0.136
Zephyr	4,298	703,727	228,739	9.93B	0.099

Table 6 — Real-world workload characteristics (results/corpus_characterization.csv). Associations shown across three corpora only — not sufficient to establish causation.

4.8 Discussion

The results indicate a genuine memory-latency trade-off rather than an unconditional win. On synthetic datasets, BUDGET-SYM reduces tracked memory relative to both baselines on every dataset tested, with the largest gains on prefix-similar and nested-scope workloads. Real-world evaluation on FreeRTOS, Arduino, and Zephyr does not confirm that this synthetic-data gain transfers to real embedded codebases: measured physical heap is consistently higher for BUDGET-SYM than Conventional on every real corpus evaluated. Lookup latency remains higher than the conventional baseline on every dataset

measured, though the lookup cache measurably reduces the project's own lookup latency relative to itself without the cache.

5. Conclusion and Scope for Future Work

This work reports BUDGET-SYM, an adaptive, scope-aware symbol-table architecture for memory-constrained embedded compilers, combining a unified INLINE/INTERNEDED/COMPRESSED representation policy with systematic threshold optimization, real-world corpus evaluation, multi-seed statistical validation, a lookup cache, and an online threshold predictor.

The proposed methodology's advantage is that representation selection is driven jointly by memory pressure, scope lifetime, and access frequency, validated with 95% confidence intervals and tested for generalization against real embedded corpora rather than synthetic data alone.

The prototype is implemented and benchmarked entirely in header-only C++14, using a custom deterministic tracked-memory accounting harness and real vendored embedded C corpora (FreeRTOS, Arduino core, Zephyr RTOS).

Results and achievement: a mean tracked-memory reduction of 1.40x-2.60x on eight synthetic benchmark datasets relative to the conventional baseline ($p < 0.01$, 30-seed validation); a grid-search threshold optimization achieving 1.87x mean compression ratio; and a 64-entry LRU lookup cache reducing the project's own cold-lookup latency by 12%-33% on five of eight synthetic datasets. The same evaluation honestly reports that on real-world corpora, BUDGET-SYM currently uses more physical heap than the conventional baseline, tracing this to unpruned persistent lookup registries — a primary finding, not a limitation to be minimized.

Scope for future work includes: (1) scope-scoped pruning or reference-counted eviction of the persistent lookup registries identified as the dominant real-world memory cost; (2) re-running the grid search directly against real embedded corpora; (3) charging the lookup cache's own memory cost against the tracked-memory model; (4) multi-seed statistical validation of the real-world corpus results; and (5) integration with a real compiler front end.

6. References

- [1] Linköping University, "Lab 2: Symbol Table — TDDE66 Compiler Construction."
- [2] D. W. Jones, "Compiler Construction — The Symbol Table," Univ. of Iowa compiler notes.
- [3] Rust compiler ecosystem, "Interning / Symbol Table" pattern documentation.
- [4] GNU Project, "MSP430 Options," GCC documentation.
- [5] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Pearson, 2006.
- [6] IEEE Author Center Conferences, "Structure Your Paper."
- [7] IEEE Author Center Conferences, "Authoring Tools and Templates."
- [8] P. Celis, P.-A. Larson, J. I. Munro, "Robin Hood Hashing," FOCS 1985, pp. 281-288.
- [9] E. Fredkin, "Trie Memory," Comm. ACM, vol. 3, no. 9, pp. 490-499, 1960.

- [10] A. Appleby, "MurmurHash3 and the SMHasher hash function test suite."

Author Contributions

All four authors contributed to design, implementation, and evaluation of BUDGET-SYM as part of a joint research effort at SCOPE, VIT Vellore.

Data and Code Availability

The BUDGET-SYM prototype, benchmark harness, and all result CSVs referenced in this manuscript are available in the project repository, with real-world corpora vendored locally.

Formatting note: internal research manuscript prepared for review; not submitted to or accepted by any venue.